# Implementation of Algorithm 1 from the paper "Approximate Optimizers and an Algorithm for Bounded Convex Set Optimization Problems"

This section describes the implementation of Algorithm 1 in the Python package `epsopt` (available at `https://github.com/rebec15/epsopt`). The purpose of the implementation is to provide a computational realization of the inner approximation procedure described in the paper. The overall software structure and the main implementation choices are presented here.

The package `epsopt` separates the mathematical representation of a convex set-valued optimization problem from the implementation of the optimization algorithm. The main components are the classes `Graph`, `Approximation`, `CP`, `IP`, `CSOP` and `ConvexSetApproximator`.

The class `Graph` represents the graph of the set-valued mapping $F$. A graph object stores the dimensions of the decision space, value space and projection variables, the generators of the common recession cone of the values as well as the graph description via a collection of convex `cvxpy` constraint functions. An object of the class `Graph` stores the constraint functions as callback functions rather than as `cvxpy`-constraints themselves. This is necessary because the same mathematical graph description must be instantiated repeatedly with different `cvxpy` variables and parameters. In particular, in each iteration of the algorithm, a problem of the form $(CP(F, w, I))$ is solved with a growing set of vertices of $I$. To ensure that the constraint $I \subseteq F(x)$ in the problem is satisfied, we check

$$(x, y)^T \in \operatorname{gr} F \quad \forall y \in \operatorname{vert} I.$$

This is sufficient because all values $F(x)$ are convex by Assumption (A2), and $I = \operatorname{conv}(\operatorname{vert} I) + G(0)$ with recession cone $G(0)$. Thus, in each iteration, a separate copy of the graph constraints is required for each new vertex of the polyhedral approximation. That is, the corresponding constraints must be generated dynamically. Since `cvxpy`-constraints are tied to the specific `cvxpy`-variables with which they are created, the graph constraints cannot simply be stored once and reused as fixed constraints. Instead, the registered constraint functions serve as reusable constraint builders that generate the appropriate `cvxpy`-constraints for each optimization problem.

Since the package uses `cvxpy` to formulate and solve the convex optimization problems, the graph constraints must be specified in a form compatible with disciplined convex programming (DCP). For further details, see [8]. The class `Graph` contains a function to validate the DCP-compliance of the supplied constraints. Furthermore, the class provides a function to check the feasibility of the given (SOP) by solving a feasibility problem over $\operatorname{gr} F$. There is also a possibility to check Assumptions (A3) and (A5). Boundedness in the sense of

$$F(x) \subseteq B + G(0) \quad \forall x \in \operatorname{dom} F$$

for some nonempty compact set $B$ is tested by checking the equivalent condition

$$\operatorname{Img} F \subseteq B + G(0).$$

This is done by solving a weighted-sum scalarization over $\operatorname{Img} F$ for all generators of $G(0)^\circ$.

By checking the rank of the generator matrix of $G(0)$, the implementation can determine whether $\operatorname{int} G(0) \neq \emptyset$, $G(0) = \{0\}$ or the necessary assumptions are not satisfied. The polyhe-

drality of $G(0)$ is implicitly ensured in the implementation since the generators of $G(0)$ must be provided by the user when initializing a graph object.

The class `Approximation` stores and updates the polyhedral inner approximation during the algorithm via its vertices as well as a minimal system of unit outer normals of the approximation. When a new vertex is added to the approximation, the unit facet outer normals are updated automatically while preserving the facet outer normals of the recession cone $G(0)$.

The convex sub-problem $(CP(F, w, I))$ (part of the core algorithm) and the inner-point sub-problem $(IP(F, \bar{x}, p, d))$ (part of the initialization algorithm) arising in Algorithm 1 are implemented as the classes `CP` and `IP`, respectively.

The class `CSOP` implements Algorithm 1. The function `computeEpsOptimizer` is used to execute the algorithm. Unlike in the description in the paper, feasibility of the (SOP) is tested whenever the user does not provide an initial point $y \in \operatorname{Img} F$. Furthermore, the procedure checks whether

$$G(0) = \mathbb{R}^q.$$

In this case, all points in the domain are optimizers and the approximation procedure need not be executed. Thus, the implementation returns the initial point $x$ from line 3 of Algorithm 1. The function used to find such an initial point solves a feasibility problem over $\operatorname{gr} F$. More precisely, for either a user-specified point $y \in \operatorname{Img} F$ or a randomly selected point $y \in \operatorname{Img} F$, the problem

$$\max_x \ 0 \quad \text{s.t.} \quad (x, y)^T \in \operatorname{gr} F$$

is solved. The initial approximation is constructed using Algorithm 5 as described below in Section 9. Note that it differs from the initialization presented in the paper. To check whether the point $y^*$ obtained from a solution of $(CP(F, w, I))$ belongs to $I + \frac{\varepsilon}{2}\mathbb{B}$, the following projection problem is solved:

$$\min_{\lambda,\mu} \ \left\| y^* - (V^T\lambda + G^T\mu) \right\| \quad \text{s.t.} \quad \begin{cases} \lambda \geqslant 0 \\ \bar{e}^T\lambda = 1 \\ \mu \geqslant 0 \end{cases}$$

where the rows of $V$ contain the vertices of $I$ and the rows of $G$ contain the generators of the recession cone $G(0)$.

The class `CSOP` also provides an option to compute a bound for the constant $\gamma$ appearing in the output of Algorithm 1. This bound is computed using the representation of $\gamma(E)$ given in the statement of correctness. The implementation solves the corresponding second-order cone program for a finite set of generators $E$ of $G(0)^\circ$. This set is computed from the user-specified generators of $G(0)$. The second-order cone problem solved is the following:

$$\max \sum_{e \in E} \lambda_e \quad \text{s.t.} \quad \begin{cases} \|\sum_{e \in E} \lambda_e e\| \leqslant 1 \\ \lambda_e \geqslant 0 \quad \forall e \in E \end{cases}$$

A relevant special case for the implementation is

$$G(0) = \{0\}.$$

The implementation remains applicable in this setting. However, the constant $\gamma(E)$ derived in the statement of correctness depends on the interior of $G(0)$ being nonempty. Therefore, the

resulting value of $\gamma(E)$ is not meaningful in this special case. Nevertheless, the small numerical examples in `epsopt/examples` suggest that the inner approximation remains meaningful.

It is worth noting that, in the implementation, the numerical tolerance

$$\delta = \frac{\varepsilon}{4}$$

is used for the solver termination criteria, including primal and dual feasibility and, where applicable, the duality gap. This choice is made to ensure sufficient numerical stability when solving the convex sub-problems. It is important to emphasize that this numerical tolerance does not directly correspond to the $\delta$-approximate solutions required in the theoretical description of Algorithm 1. In the algorithmic analysis in the paper, a $\delta$-solution of a convex sub-problem is defined as a point whose objective value differs from the optimal value by at most $\delta$. Convex solvers, however, generally do not control this objective-value deviation explicitly. Instead, they terminate based on numerical residuals associated with the Karush–Kuhn–Tucker conditions, such as primal feasibility, dual feasibility, and the primal-dual gap.

In practice, the convex sub-problems arising in Algorithm 1 can be numerically delicate. When using the default solver tolerances of `Clarabel` and `SCS`, we observed numerical instabilities, including inaccurate termination, incorrect infeasibility reports, and inconsistent duality-gap estimates. These issues may lead to premature termination or incorrect acceptance of new points during the approximation procedure. For this reason, the solver tolerances are set to $\delta$. This choice provides a numerical tolerance that is sufficiently small relative to the geometric accuracy $\varepsilon$ required by the algorithm.

Finally, the class `ConvexSetApproximator` implements the special application of Algorithm 1 to the approximation of closed convex sets $S$. The function `approximate` generates the graph of a set-valued problem of the following form:

$$\operatorname{gr} F := \left\{ (0, x, y)^T \in \mathbb{R}^{1+n+q} \,\middle|\, (x, y)^T \in S \right\}.$$

Thus,

$$F(z) = \begin{cases} S & z = 0 \\ \emptyset & \text{otherwise.} \end{cases}$$

The function then uses Algorithm 1 implemented in `CSOP` to obtain a $\gamma\varepsilon$-H-approximation of the set $S$.

The package examples contain tests for the algorithm, including an application to the portfolio optimization example from the paper as well as tests for the class `ConvexSetApproximator`. The file `portfolio_opt_scenarios.py` contains the class `PortfolioOptScen` which can be used for examples of the form introduced in the paper. The class contains functions to construct a graph based on the supplied parameters, run the algorithm, and generate plots in the two-dimensional case ($q = 2$). The `main` method contains the instance presented in the paper, including the visualization of values $F(x)$ for a user-specified value of $x$ using `ConvexSetApproximator`.

## Implemented Initialization Algorithm

To initialize the inner approximation $I$ in Algorithm 1, we require a procedure that returns $\operatorname{int} I_0 \neq \emptyset$ for the initial approximation $I_0$ as well as $0^+ I_0 = G(0)$. As $\operatorname{int} G(0)$ is assumed to be nonempty (Assumption A5), for $P_0 \neq \emptyset$ we directly obtain $\operatorname{int} I_0 \neq \emptyset$ if $I_0 = P_0 + G(0)$. Moreover, the initial approximation must satisfy $\bar{y} \in I_0$ in order to ensure $\bar{y} \in F(\bar{x})$ on

termination. Algorithm 6 provides such an initialization.

**Algorithm 6:** Computation of an initial polyhedral inner approximation $I$ of a closed convex set $F(\bar{x})$ with $\bar{y} \in F(\bar{x})$, $\operatorname{int} I \neq \emptyset$ and $0^+ I = G(0)$ for closed convex set-valued mappings $F$.

**Input:** Closed convex set-valued mapping $F$ with $G(0) \subsetneq \mathbb{R}^q$ polyhedral and $\operatorname{int} G(0) \neq \emptyset$, point $\bar{x} \in \operatorname{dom} F$, point $\bar{y} \in F(\bar{x})$.

**Output:** bounded portion $P$ of an initial, nonempty, full-dimensional polyhedral inner approximation $I = P + G(0)$ of $F(\bar{x})$ in $\mathcal{V}$-representation with $\bar{y} \in P$.

1 Compute $y^* \in -G(0)^\circ \setminus \{0\}$
2 Compute $p \in \operatorname{int} F(\bar{x})$
3 $d^0 \leftarrow \dfrac{y^*}{\|y^*\|}$
4 Complete $d^0$ to an orthonormal basis of $\mathbb{R}^q$ with vectors $\{u^1, \ldots, u^{q-1}\}$
5 **for** $i = 1, \ldots, q-1$ **do**
6 $\quad d^i \leftarrow -d^0 + u^i$
7 **end**
8 $d^q \leftarrow -d^0 - \sum_{i=1}^{q-1} u^i$
9 $P \leftarrow \{p, \bar{y}\}$
10 **for** $d \in \{d^1, \ldots, d^q\}$ **do**
11 $\quad$ find optimal solution $(t, y)$ of $(IP(F, \bar{x}, p, d))$
12 $\quad P \leftarrow \operatorname{conv}(P \cup \{y\})$
13 **end**
14 **return** $P$

We provide some details on Algorithm 6. The algorithm returns a bounded polyhedron $P$ such that $I = P + G(0)$ is an inner approximation of some given value $F(\bar{x})$. The direction $y^*$ in line 1 can be found by solving a convex program. The polyhedral cone $G(0)$ can be represented by $G(0) = \operatorname{cone}\{c^1, \ldots, c^k\}$. The problem of finding some direction $y^* \in -G(0)^\circ$ can thus be formulated as follows:

$$\max_{y \in \mathbb{R}^q} \sum_{i=1}^{k} (c^i)^T y \quad \text{s.t.} \quad \begin{cases} (c^i)^T y \geqslant 0 & \forall i = 1, \ldots, k \\ \|y\| \leqslant 1 \end{cases}$$

The problem always has an optimal solution as the feasible set is compact due to the constraint $\|y\| \leqslant 1$ and nonempty as $-G(0)^\circ$ is nonempty. Since we assume $G(0) \subsetneq \mathbb{R}^q$, $-G(0)^\circ$ is strictly larger than $\{0\}$ and the direction $y^*$ returned from the optimization problem above is not 0.

To find an interior point $p$ of $F(\bar{x})$ in line 2, consider that $\operatorname{int} G(0)$ was assumed nonempty which implies $\operatorname{int} F(\bar{x}) \neq \emptyset$. Hence, such a point exists. To compute an inner direction $c$ of $G(0)$, consider that $G(0) = \operatorname{cone}\{c^1, \ldots, c^k\}$ is a polyhedral cone. Thus, it follows from

$$\operatorname{int} G(0) = \operatorname{int} \left\{ \sum_{i=1}^{k} \lambda_i c^i \,\middle|\, \lambda_i \geqslant 0 \quad \forall i = 1, \ldots, k \right\} = \left\{ \sum_{i=1}^{k} \lambda_i c^i \,\middle|\, \lambda_i > 0 \quad \forall i = 1, \ldots, k \right\} \text{[9, p. 45]},$$

that

$$c := \frac{1}{k} \sum_{i=1}^{k} c^i \in \operatorname{int} G(0).$$

Since $\bar{y} \in F(\bar{x})$, we obtain $\bar{y} + c \in \operatorname{int} F(\bar{x})$.

Given $y^* \in -G(0)^\circ \setminus \{0\}$, $G(0) \subseteq \{x \in \mathbb{R}^q \mid {y^*}^T x \geqslant 0\}$ by the definition of the polar cone.

In lines 4–8 of Algorithm 6, the direction $d^0$ is completed to an orthonormal basis of $\mathbb{R}^q$ with vectors in $\mathbb{R}^q \setminus G(0)$. The vectors $u^1, \ldots, u^{q-1}$ can be obtained by completing $d^0$ to a basis and applying an orthogonalization procedure such as the Gram-Schmidt procedure. These vectors are then used to construct vectors $d^i$ in

$$H = \{x \in \mathbb{R}^q \mid y^{*T} x < 0\}.$$

Upon completion of line 8, we have

$$y^{*T} d^i = y^{*T}(-d^0) + y^{*T} s \quad \forall i = 1, \ldots, q$$

for $s$ being some part of $d^i$ which is orthogonal to $y^*$. Thus,

$$y^{*T} d^i = -\|y^*\| < 0 \quad \forall i = 1, \ldots, q$$

and $d^i \in H \subseteq \mathbb{R}^q \setminus G(0)$.

The problem $(IP(F, \bar{x}, p, d))$ solved in line 11 has the following form: The second auxiliary problem is an inner-point problem. It is used to generate an initial inner approximation of the set $F(\bar{x})$.

$$\max\ t \quad \text{s.t.} \quad \begin{cases} (\bar{x}, y)^T \in \operatorname{gr} F \\ y = p + td \end{cases} \qquad (IP(F, \bar{x}, p, d))$$

In the initialization, a point $p \in F(\bar{x})$ and a direction $d$ are chosen. The problem $(IP(F, \bar{x}, p, d))$ determines the maximal step length $t$ such that the point $p + td$ remains in $F(\bar{x})$. The problem admits a solution for every chosen $d$ as $d \notin G(0) = 0^+[F(\bar{x})]$. Since $\operatorname{gr} F$ is assumed to be closed and convex, $F(\bar{x})$ is closed and convex. Hence, we cannot move arbitrarily far in direction $d$ from point $\bar{y} \in F(\bar{x})$ if $d$ is not in the recession cone of $F(\bar{x})$. Since $F(\bar{x})$ is closed, the optimal value of $(IP(F, \bar{x}, p, d))$ is attained at a boundary point $y$ of $F(\bar{x})$.

Finally, the initial inner approximation $I = P + G(0)$ constructed from the output $P$ of Algorithm 6 has full dimension $q$. Since $P$ is nonempty, this follows directly from $I = P + G(0)$ and $\operatorname{int} G(0) \neq \emptyset$ (Assumption (A5)).